Module 4

Topics: Categorical data, Performing arithmetic operations, SAS dates and date functions

Files to download: mod4.sas7bdat

4.1 - Creating categorical variables in SAS

When performing a stratified analysis in epidemiology we need to have variables in which the continuous data (e.g., age, weight) have been grouped. We call variables that contain such grouped data categorical variables. In order to create categorical variables we need to 1) determine the distribution of the continuous data, 2) make a decision about how many categories we want in our categorical variable, 3) determine the "cut-point" values of the continuous variable that will allow us to equally divide the population between these levels and 4) create the categorical variable in SAS using a series of IF/THEN statements that assign values to the categories based on the cut-point values.

The SAS language for operators used when creating categorical variables:

Greater than:	Use **>** or **GT**	Less than:	Use **<** or **LT**
Greater than or equal to:	Use **>=** or **GE**	Less than or equal to:	Use **<=** or **LE**

This is an overview of SAS operators for reference.
http://support.sas.com/documentation/cdl/en/lrcon/62955/HTML/default/viewer.htm#a000780367.htm

Using the mod4 dataset, let's look at the data in the variable WEIGHT that has continuous numeric data on subjects' weight.

```
libname NICOLE 'H:\AEPI537D';

PROC UNIVARIATE DATA=NICOLE.mod4;
      VAR WEIGHT;
RUN;
```

Let's say that you want to create a 2-level categorical variable for weight that has approximately equal numbers of observations in each group. You can use the output of the UNIVARIATE procedure to determine the cut-point which in this case will be the value of weight for which one-half of the subjects weighed less and the other half weighed that much or greater. The quantile that provides this value is the median which is given as part of the standard output of the UNIVARIATE procedure. For the mod4 dataset the median is 135 pounds. The following program creates a two-level variable called WT2LEV that has the value 0 if the subject's weight is less than or equal to 135 pounds and the value 1 if the weight is greater than 135 pounds.

```
DATA TEMP;
      SET NICOLE.mod4;
            IF 0 < WEIGHT <= 135 THEN WT2LEV=0;
            IF WEIGHT > 135 THEN WT2LEV=1;
RUN;
```

```
PROC FREQ;
      TABLES WT2LEV;
RUN;
```

Notice that this program correctly assigns a missing value to WT2LEV for any subject who has a missing value for weight by putting a lower bound on the first level. This is necessary because SAS stores missing values as an extremely small negative number.

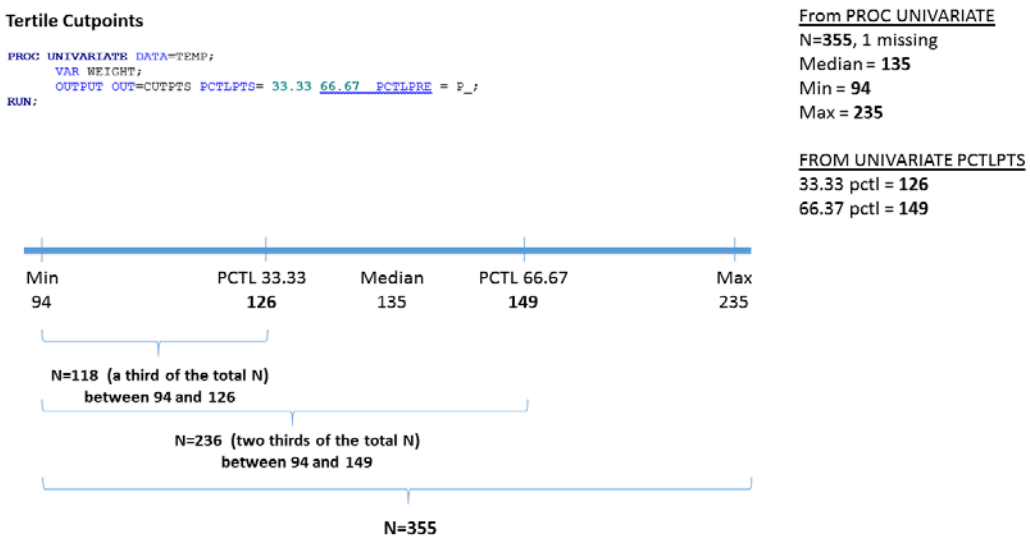
As another example, let's say that you want to create a 3-level variable for weight that has approximately equal numbers of observations in each group. There are at least two ways that you could go about doing this. The first is to use the UNIVARIATE procedure to determine the tertile cut-points, then to create the new variable using IF/THEN statements as usual. Because UNIVARIATE does not provide the tertile cutpoints by default, we must use an OUTPUT statement and specify the percentile values (PCTLPTS) and a prefix (P_) that SAS can use to name the variable in the output dataset that contains the cut-point values. The following code illustrates the use of the UNIVARIATE procedure to generate the tertile cut-points.

```
PROC UNIVARIATE DATA=TEMP;
      VAR WEIGHT;
      OUTPUT OUT=CUTPTS PCTLPTS= 33.33 66.67  PCTLPRE = P_;
RUN;
```

Let's take a look at the CUTPTS output dataset.

```
PROC PRINT DATA= CUTPTS;
RUN;
```

The dataset has one observation and 2 variables. The variables are named P_33_33 and P_66_67 which is a combination of the percentile cut-points and the prefix that you provided above. These variables contain the cut-points for the tertile of the weights, namely 126 pounds and 149 pounds.



Next we will create the variable WT3LEV using IF/THEN statements as follows:

```
DATA TEMP2;
      SET TEMP;
            IF WEIGHT > 0 AND WEIGHT <= 126 THEN WT3LEV=0;
            IF WEIGHT > 126 AND WEIGHT <= 149 THEN WT3LEV=1;
            IF WEIGHT > 149 THEN WT3LEV=2;
RUN;

PROC FREQ DATA= TEMP2;
      TABLES WT3LEV;
RUN;
```

Remember that when you are creating categorical variables it is up to you to tell SAS what you want done with any observations with missing data for the continuous variable. In the previous examples we have been using a lower limit for the first level of the categorical variable so that subjects with missing data would not be categorized at the lowest level but would be correctly assigned a missing value in the new categorical variable. This also could be done by specifically assigning the new categorical variable a value of missing when weight is missing. However, this statement must come last to be effective as in the following alternative program:

```
DATA ONE;
      SET TEMP2;
            IF WEIGHT <= 126 THEN WT3LEV2=0;
            IF WEIGHT > 126 AND WEIGHT <= 149 THEN WT3LEV2=1;
            IF WEIGHT > 149 THEN WT3LEV2=2;
            IF WEIGHT = . THEN WT3LEV2=. ;
RUN;
```

Let's take a look at the distribution of the WT3LEV2 variable to make sure the result is the same as in the last program.

```
PROC FREQ DATA=ONE;
      TABLES WT3LEV*WT3LEV2/norow nocol nopercent;
RUN;
```

Another way that you can create a new categorical variable with any number of categories you wish is to use the RANK procedure. The following example shows how to use this procedure.

```
PROC RANK DATA=TEMP2 OUT=TWO GROUPS=3;
      VAR WEIGHT;
      RANKS WT3LEV3;
RUN;

PROC FREQ DATA=TWO;
      TABLES WT3LEV3;
RUN;
```

4.2 - Performing arithmetic operations in SAS

There are several variables in this dataset that store numeric data that might be more useful to us if it were in another form. For example, we have two variables that store data about birth weight; BIRTHLB stores the pounds value and BIRTHOZ stores the ounces value. Although it was necessary to have two variables to store the data in the data set, having the birth weight and height spread across two variables makes it impossible to perform any analyses on these measures. Most studies involving birth weight use grams as the unit of measure. In epidemiology, we often use body mass index (BMI) to look at a person's weight for height. We can create a variable for BMI from the variables for weight, HEIGHTFT and HEIGHTIN with the appropriate metric conversions. SAS provides the ability to perform arithmetic operations on your data.

The SAS language for arithmetic operations is as follows:

Addition:	Use **+**	Example: x=y+z
Subtraction:	Use **-**	Example: x=y-z
Multiplication:	Use *****	Example: x=y*x
Division:	Use **/**	Example: x=y/z
Exponentiation:	Use ******	Example: x=y**z This raises y to the z power

In order to convert your variables of birth weight pounds and birth weight ounces to one variable of birth weight in grams you need to know that 1 pound = 16 ounces and 1 ounce = 28.35 grams. The formula for BMI is weight (kg)/height (m)**2.

The following programming steps will add new variables for birth weight in grams (bw_gm) and BMI to our permanent data set. At the same time, we will create a categorical variable for birth weight of <= 2500 grams vs >2500 grams.

```
DATA TEMP3;
      SET TEMP2;

      BW_OZ = BIRTHLB*16 + BIRTHOZ;
      BW_GM = BW_OZ*28.35;

      IF BW_GM <=2500 THEN BW_GM1 = 1;
      IF BW_GM >2500 THEN BW_GM1 = 0;
      IF BW_GM = . THEN BW_GM1 = .;

      HT_IN = HEIGHTFT*12 + HEIGHTIN;*converts feet and inches to height in inches;
      WEIGHTKG = WEIGHT * 0.4536;     *converts weight in pounds to weight in kilograms;
      HEIGHTM = HT_IN *0.0254;        *converts height in inches to height in meters;
      BMI = WEIGHTKG/HEIGHTM**2;      *body mass index;
RUN;
```

Let's take a look at the values for these variables for a few of the observations to make sure that our program worked the way that we intended it to. It is always a good idea after you have performed arithmetic operations on your data to make sure that the program was correct and that your results make sense.

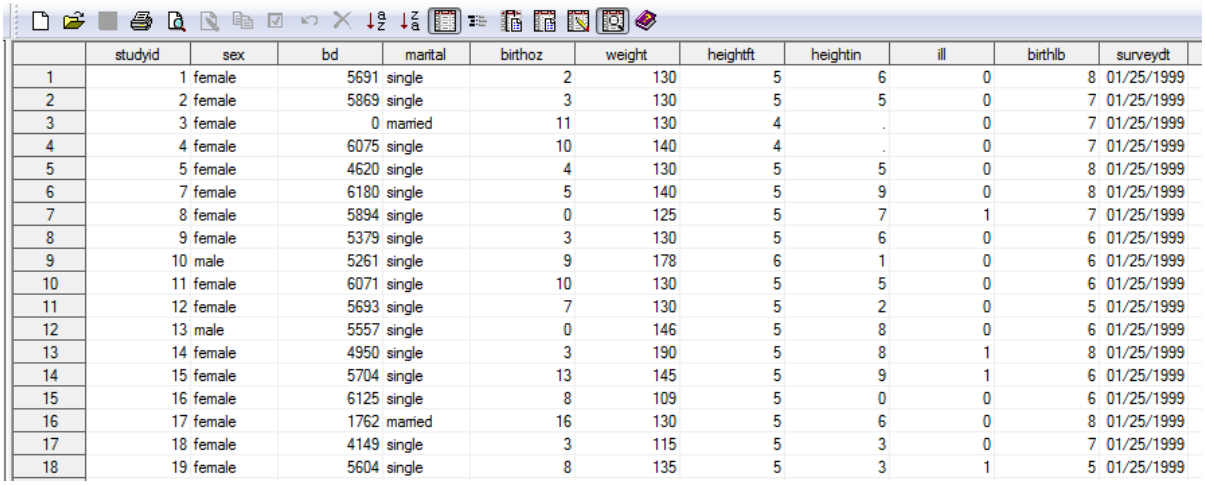
```
PROC PRINT DATA=TEMP3 NOOBS;
      VAR BIRTHLB BIRTHOZ BW_OZ BW_GM BW_GM1 WEIGHT WEIGHTKG HEIGHTFT HEIGHTIN
          HT_IN HEIGHTM BMI;
RUN;
```

Notice that the values for new variables that were created from arithmetic operations on variables with missing data are correctly set to missing. When performing arithmetic operations, SAS will automatically account for missing values in your data. This is in contrast to the situation in which you are creating categorical variables from numeric data and you must be careful to correctly account for missing values in your program.

4.3 - Working with dates in SAS

You have two date variables in your data set, BD (birthdate) and SURVEYDT (interview date). SAS knows these are dates because the *informat* mmddyy10. was specified when the data were entered or imported into SAS. The SAS system stores dates as single, unique numbers so you can use them in programs like any other numeric variable.

Let's take a look at these two variables in the dataset by using the VIEWTABLE feature of the SAS Explorer.



	studyid	sex	bd	marital	birthoz	weight	heightft	heightin	ill	birthlb	surveydt
1	1	female	5691	single	2	130	5	6	0	8	01/25/1999
2	2	female	5869	single	3	130	5	5	0	7	01/25/1999
3	3	female	0	married	11	130	4	.	0	7	01/25/1999
4	4	female	6075	single	10	140	4	.	0	7	01/25/1999
5	5	female	4620	single	4	130	5	5	0	8	01/25/1999
6	7	female	6180	single	5	140	5	9	0	8	01/25/1999
7	8	female	5894	single	0	125	5	7	1	7	01/25/1999
8	9	female	5379	single	3	130	5	6	0	6	01/25/1999
9	10	male	5261	single	9	178	6	1	0	6	01/25/1999
10	11	female	6071	single	10	130	5	5	0	6	01/25/1999
11	12	female	5693	single	7	130	5	2	0	5	01/25/1999
12	13	male	5557	single	0	146	5	8	0	6	01/25/1999
13	14	female	4950	single	3	190	5	8	1	8	01/25/1999
14	15	female	5704	single	13	145	5	9	1	6	01/25/1999
15	16	female	6125	single	8	109	5	0	0	6	01/25/1999
16	17	female	1762	married	16	130	5	6	0	8	01/25/1999
17	18	female	4149	single	3	115	5	3	0	7	01/25/1999
18	19	female	5604	single	8	135	5	3	1	5	01/25/1999

Internally SAS stores the date as a numeric value representing the number of days between January 1, 1960 and the date that was entered. Dates before January 1, 1960 are negative numbers. Those after January 1, 1960 are positive numbers. You will notice that SAS may show dates as either this internal number (e.g., BD) or as a more recognizable formatted date (e.g., SURVEYDT) depending upon how the data were formatted when entered into SAS.

Because SAS treats date values as numbers, you can sort them, determine time intervals, and perform calculations on them. A common use of date variables in epidemiology is the determination of age at

the time of enrollment into a study. The following program will determine age at the time of interview. We saw some of these date functions in a previous lab.

```
DATA TEMP4;
      SET TEMP3;
         CALC_AGE = (SURVEYDT - BD)/365.25; *Calculates the exact age at time of interview;
         ENTRYAGE = INT(CALC_AGE);  *Gives the integer value of age at time of interview;
RUN;

      PROC PRINT DATA=TEMP4 NOOBS;
         VAR STUDYID BD SURVEYDT CALC_AGE ENTRYAGE ;
         FORMAT BD MMDDYY10.; *Specifies how you want the dates displayed;
      RUN;
```

The variable ENTRYAGE is the integer value of the calculated age.

The argument in the example above is the value stored in the variable CALC_AGE for each observation. The program below creates a new variable STARTYR which is the year that the person completed the survey.

```
DATA TEMP5;
      SET TEMP4;
            STARTYR = YEAR(SURVEYDT);
RUN;

      PROC FREQ DATA=TEMP5;
         TABLES STARTYR;
      RUN;
```

This gives us the number of persons who took our survey in each year. The day and month functions operate the same way to extract the day and month out of the argument date value, respectively.

Previously we saw the DATE() function which contains no argument. This function returns today's date and the following program illustrates how this function could be used to calculate the current follow-up time for a person since they completed the survey.

```
DATA TEMP6;
      SET TEMP5;
            TDAY = DATE();
            FUT = (TDAY - SURVEYDT)/365.25;
RUN;
```

Another useful date function we saw previously is MDY(MM,DD,YYYY). The letters in the argument stand for the 2-digit month, 2-digit day, and 4-digit year that represent the date of your choice. For example, in the last program you might like to calculate the follow-up time that each person will have at the end of 2005 when the study you are conducting ends. The following program will perform this calculation.

```
DATA FUT2;
      SET TEMP6;
            ENDDT = MDY(12,31,2005);
            FUT2 = (ENDDT - SURVEYDT)/365.25;
RUN;

PROC PRINT DATA=FUT2;
      VAR ENDDT SURVEYDT FUT2;
      FORMAT ENDDT MMDDYY10.;
RUN;
```

4.4 - Performing multiple operations in the same DATA step

During this SAS session we have created multiple temporary SAS datasets (TEMP to TEMP6) by performing just one operation in each data step. Keep in mind that you could have performed all of the operations in this lab that required 7 temporary SAS datasets (TEMP through TEMP6) in *one* data step as shown below.

```
DATA TEMP;
      SET NICOLE.mod4;
            IF 0 < WEIGHT <= 135 THEN WT2LEV=0;
            IF WEIGHT > 135 THEN WT2LEV=1;

            IF WEIGHT GT 0 AND WEIGHT LE 126 THEN WT3LEV=0;
            IF WEIGHT GT 126 AND WEIGHT LE 149 THEN WT3LEV=1;
            IF WEIGHT GT 149 THEN WT3LEV=2;

            BW_OZ = BIRTHLB*16 + BIRTHOZ;
            BW_GM = BW_OZ*28.35;

            IF BW_GM <=2500 THEN BW_GM1 = 1;
            IF BW_GM >2500 THEN BW_GM1 = 0;
            IF BW_GM = . THEN BW_GM1 = .;

            HT_IN = HEIGHTFT*12 + HEIGHTIN;
            WEIGHTKG = WEIGHT * 0.4536;
            HEIGHTM = HT_IN *0.0254;
            BMI = WEIGHTKG/HEIGHTM**2;

            CALC_AGE = (SURVEYDT - BD)/365.25;
            ENTRYAGE = INT(CALC_AGE);
            STARTYR = YEAR(SURVEYDT);
            TDAY = DATE();
            FUT = (TDAY – SURVEYDT)/365.25;
RUN;
```

Combining your operations in one data step takes advantage of the fact that you can use newly created variables within the same data step in SAS. This is a somewhat more efficient way to write programs but it is completely up to you how you would like to perform your operations in SAS. Of course you would still want to write a program that would allow you to check to make sure that each of the operations in your data step were working correctly as demonstrated in the previous pages. If

you would like to be able to check each block of code as you write it, I recommend adding a few lines at a time and re-running the data step as you go.

4.5 - IF, ELSE IF and ELSE statements

One alternative to using a series of IF statements is to use IF in combination with ELSE IF and ELSE statements. Although IF statements are adequate for most projects, sometimes ELSE IF and ELSE statements will help to simplify complicated code.

IF, ELSE IF and ELSE statements are used together as a block of code (which we will call the 'IF-ELSE' block). In an IF-ELSE block, SAS only reads and executes the *first* line of the block that matches the data that it is reading. Compare this to sets of IF statements, where SAS reads and executes *every* line of code.

The following example will demonstrate how the IF-ELSE block works by showing the same code first using IF statements and the second using the IF-ELSE block. In such a simple situation, using either method would be just as easy. The IF-ELSE block becomes particularly useful when the coding is more complicated. Also, if you have a very large dataset, using an IF-ELSE block can cut down on the amount of time it takes SAS to run the program since it has to read and execute less code than using only IF statements.

Here is the original code (from a previous example above):

```
IF WEIGHT <= 126 THEN WT3LEV2=0;
IF WEIGHT > 126 AND WEIGHT <= 149 THEN WT3LEV2=1;
IF WEIGHT > 149 THEN WT3LEV2=2;
IF WEIGHT = . THEN WT3LEV2=.;
```

Using the IF-ELSE block we do not need the lower bound restrictions because SAS will choose *only* the first line in the block that matches the observation it is processing:

```
IF WEIGHT = . THEN WT3LEV2 = .;
   ELSE IF WEIGHT <= 126 THEN WT3LEV2 = 0;
   ELSE IF WEIGHT <= 149 THEN WT3LEV2 = 1;
   ELSE WT3LEV2 = 2;
```

For example, if SAS is processing an observation with WEIGHT = 140, the first line in the block that matches is ELSE IF WEIGHT LE 149 THEN WT3LEV2 = 1. For an observation with WEIGHT = 200, the first line that matches is ELSE WT3LEV2 = 2. The results are the same using IF statements and the IF-ELSE block.

Notice that because SAS chooses only the first line that matches, we had to move our code for missing values to the beginning of the IF-ELSE block (since . is less than 125). When using an IF-ELSE block you must make sure your lines of code are in the proper order so that SAS will always choose the right one first.

Rules for using IF-ELSE blocks of code

- The first line of the block must start with IF
- Each line of code starting with IF is a new IF-ELSE block
- Every other line in the block must start with ELSE (either ELSE IF or ELSE)
- The format is IF… ELSE IF (as many or few as needed)… ELSE
- You do not need an ELSE statement (you can just use IF and ELSE IF)
- You do not need an ELSE IF statement (you can just use IF and ELSE)
- There can be a maximum of one ELSE statement (but as many ELSE IF as needed)
- The statements must be in the proper order you want SAS to choose from (remember that SAS will only choose the first line of code that matches the observation it is processing)

Examples of common mistakes when using IF-ELSE blocks

1) Forgetting to use **ELSE** IF in the middle lines of an IF-ELSE block:

```
IF WEIGHT = . THEN WT3LEV2 = .;
IF WEIGHT LE 126 THEN WT3LEV2 = 0;
IF WEIGHT LE 149 THEN WT3LEV2 = 1;
ELSE WT3LEV2 = 2;
```

2) Lines of code in the wrong order:

```
IF WEIGHT = . THEN WT3LEV2 = .;
ELSE IF WEIGHT LE 149 THEN WT3LEV2 = 1;
ELSE IF WEIGHT LE 126 THEN WT3LEV2 = 0;
ELSE WT3LEV2 = 2;
```

The important thing to remember about the difference between IF blocks versus and IF-ELSE blocks is that SAS will overwrite one IF statement with another, but it will not overwrite with a properly written IF-ELSE block. In the example from earlier:

```
IF WEIGHT = . THEN WT3LEV2 = .;
   ELSE IF WEIGHT <= 126 THEN WT3LEV2 = 0;
   ELSE IF WEIGHT <= 149 THEN WT3LEV2 = 1;
   ELSE WT3LEV2 = 2;
```

SAS first assigns WT3LEV2 for the missing values of WEIGHT and because the next step is an ELSE-IF, SAS will NOT ever re-assign the value of WT3LEV2 for those missing weight. It knows it has already handled the missing values and it will only deal with whichever WEIGHT values remain for the next step. Once it deals with the WEIGHT values less than or equal to 126 in the next step, it will ignore those for the next ELSE-IF, and so on.

MOD 4: USE SAS COMMENTS TO NUMBER YOUR WORK.

1. Use the UNIVARIATE procedure to find the quintile (5-level) cut-points for the variable WEIGHT. Using these cut-points, create a 5-level categorical variable called WTQUINT in a dataset called MOD4_1 created using the TEMP6 dataset we left off with earlier. Run a FREQ procedure on this variable.

2. Use the RANK procedure to create a 5-level categorical variable called WTQUINT2 in a dataset called MOD4_2 from the dataset MOD4_1.

3. The following formula can be used to calculate body mass index (BMI) directly using U.S. units rather than converting the units to their metric equivalents:

BMI = (weight in pounds / height in inches squared) x 703

a. Using this formula, create a new variable called BMI2 in a temporary SAS dataset called MOD4_3 using the MOD4_2 dataset from above. The variable for weight in pounds is called WEIGHT and the variable for height in inches is called HT_IN (we made this earlier in the lab from height in feet plus inches).

4. Create a new 4-level categorical variable called BMILEVEL from BMI2 that has cut-points corresponding to the following: underweight (BMI below 18.5); normal (BMI 18.5 – 24.9); overweight (BMI 25 – 29.9); and obese (BMI 30 or greater) and code the 4 levels as 0, 1, 2, and 3, respectively. Create this new variable in a dataset called MOD4_4 from the dataset MOD4_3 above. Perform a frequency procedure so that you can examine the counts in the new BMILEVEL variable.

5. Create a new variable called AGE_2005 that contains the integer age that each subject will be on the last day of the year 2005 in a new SAS dataset called MOD4_5 using the MOD4_4 dataset from the last question. Perform a UNIVARIATE procedure so that you can examine the distribution of the ages.

6. Using IF statements, use the MOD4_5 dataset from the previous question to create a new categorical variable called ILLSMK that is a combination of ILL and SMOKE in a dataset called MOD4_6. You can number categories however you like, just have one for each possible combination (ILL=0/SMOKE=0, ILL=0/SMOKE=1, ILL=1/SMOKE=0, ILL=1/SMOKE=1, etc.). Do a PRINT procedure to see the new variable side by side with the ones used to create it.

7. Using IF-ELSE statements, create a dataset called MOD4_7 from the MOD4_6 dataset in the previous question to create a new categorical variable called ILLWT where people with ILL=0 and WEIGHT <=130 are assigned a 1, people with ILL=1 and WEIGHT <=150 are assigned a 2, and everyone else that doesn't meet one of those conditions is assigned a 3.